MHS: A Registry-Driven, Authority-Free Hardware Layer for Agentic Software-Defined Vehicles and Other Cyber-Physical Platforms

Yuvaraja
yuvaraja.robo.ai@gmail.com

Abstract—An agent that reads or acts on hardware needs an answer to four questions: what it may ask for, how that request reaches a device, where the device is, and whether the request may proceed. Existing agent runtimes answer the first with a tool list and leave the rest to code written per device, so every new sensor becomes a new tool and a new code path, and nothing checks that a fallback measures the same thing as the path it replaces. This paper presents MHS, a hardware layer in which every hardware fact is data in a checked registry—interfaces, instances, resources, capabilities, bindings, interlock policies, lenses and controllers—and the core that serves it is generic: it knows no vehicle, no board and no pin, and a boundary test fails if one appears in its logic. MHS decides no authority. A mutation reaches a device only with an admission minted by a separate gate for that exact request digest; MHS verifies it and cannot mint one, and a device-level interlock can still refuse an admitted request. Live data leaves only through declared reductions with rate and byte budgets; anything held over time is a lease that expires to a safe state; control loops are declared in the registry and the model only sets goals. Nodes federate by importing one another as an interface with a pinned registry digest. Domain and board knowledge arrive as packs imported by a node profile, with a vehicle pack—the vehicle signal model over a data broker, the CAN bus, standard diagnostics and a service-oriented diagnostic API—as the first. We give the registry model, the compile rules, the two-lock control path, the reason codes, and measurements from a Raspberry Pi 5 bench node. The contribution is an implementable layer with its evidence, not a certified product.

Index Terms—hardware abstraction, capability registry, agentic AI, software-defined vehicle, vehicle data broker, signal model, device interlock, tool protocols

I. Introduction

Agentic systems plan, retrieve and invoke tools [1], [2]. When the tools touch hardware, the runtime inherits a set of obligations that tool schemas do not express: which device answers, how fresh the answer is, what the device may safely be told, who may ask, and what happens when the path that usually answers is absent. A companion paper [3] places these obligations in a layered vehicle architecture and makes one component, the Automotive Capability Gateway (ACG), the sole route from an AI-originated proposal to a vehicle capability. That paper defines the authority predicate. It does not define the layer beneath it: the one that opens the bus, reads the register, applies the device's own limits and reports back.

This paper is about that layer. MHS began as the adapter half of the vehicle's service and communication layers for devices that have no service of their own, and was then made domain-neutral with the vehicle as its first pack. Its name follows Anthropic's Model Hardware Standard [4], whose public description—read and write primitives, discoverable devices, a reference file stating what a device can measure, what can be adjusted and what safety limits will be enforced, and access through the Model Context Protocol (MCP) [5]—matches the shape built here. MHS is aligned with that description and claims no conformance: the standard's specification is unpublished at the time of writing, and this paper makes no statement about its wire format, schema or certification.

Three questions organise the work. RQ1: how can every hardware fact be data, so that adding a device changes a schema rather than a tool list, and so that a checker rather than a reviewer catches a fallback that measures the wrong thing? RQ2: how can a layer execute reads and writes without deciding whether they are authorised, and still refuse on its own account when a device's limits say so? RQ3: how can one core serve several nodes, boards and domains without carrying knowledge of any of them?

The contributions are (i) a registry model of eight record types whose compile rules are each tripped by a mutation test; (ii) a two-lock control path in which authorisation and device safety are separate checks with separate reason codes; (iii) lenses, leases and declared controllers as the only forms of live data and continuous control; (iv) federation by digest-pinned import; (v) a pack mechanism by which domain and board knowledge leave the core; and (vi) a bench evaluation on a Raspberry Pi 5 with a simulated vehicle, reported with its limits.

II. Background and Gap

A. The vehicle side

Software-defined vehicles expose signals and services through typed middleware: service-oriented communication management [6], a standard vehicle signal model that names every sensor and actuator in one semantic tree [7], and a data broker that serves that tree at runtime, with providers publishing values into it and consumers reading, subscribing and requesting actuations [8]. Two

properties of the broker in use shape the design. It holds one value per signal—there is no separate target and current—and an actuation is forwarded to the provider that owns the actuator: the broker "does not guarantee that the desired actuator value will be fully updated" [8]. Hence the only evidence that an actuator moved is the value its provider publishes afterwards, and a read-back after every actuation is mandatory rather than a preference.

Diagnostics run over UDS [9] carried on ISO-TP [10] or DoIP [11], and increasingly through the service-oriented SOVD API [12], [13]. Below the middleware sit device buses—CAN, I^2C, SPI, serial—and general-purpose I/O lines whose numbering on a common single-board computer depends on the kernel date [14], which is why MHS opens an I/O controller by its label [15] and never by number.

B. The agent side

MCP standardises how a host offers schema-described tools to a model [5]. Two constraints matter for a hardware surface. First, the model APIs that consume the tool list reject a oneOf at the root of a tool's input schema; one such tool makes the whole session fail [16]. Second, when the model is local and its output is constrained by a grammar generated from the schema, the common converter skips unsupported keywords silently [17], so a grammar can be looser than its schema without warning. A registry that generates tool schemas must therefore refuse the keywords that either consumer would mishandle.

C. Gap

Between typed vehicle middleware and schema-described tools there is no artefact in which device facts, semantic capabilities, fallback order, freshness bounds and device safety limits are stated once and checked together. [3] names the boundary; this paper builds the layer under it. The Model Hardware Standard announcement [4] places safety limits at the device, enforced by the driver—the same separation from authorisation that MHS makes—and confirms the direction without supplying a specification.

III. Design Principles

Figure 1 places MHS between its consumers and the wires; Table I states the model. Six principles follow from it.

The registry is data. A node profile is a set of plain declarative files; a checker validates it and emits a digest; a compiler writes a runtime database whose own constraints repeat the rules, so a hand edit fails too. The direction is one way: the declarations are authored, the database is generated and never edited.

Native identifiers never leave the adapter. A pin, register, frame identifier, signal path or diagnostic service appears in exactly one place, the resource's native block, read by the adapter and never returned. A compile rule scans every agent-visible string—descriptions, aliases, refusal sentences—for signal paths, device nodes, hexadecimal identifiers, secrets and controller labels. [3] states the invariant; the compiler enforces it.

TABLE I
The Four Questions and the Records That Answer Them

Question	Concept	File
What	capability: semantic id, type, class, parameters, outcome, freshness	capabilities.yaml
How	binding: capability → resource in priority order, with transform and unit	bindings.yaml
Where	resource on an instance of an interface, opened by an adapter	resources.yaml, interfaces.yaml
Whether	admission from the gate (authorisation) and interlock policy (device safety)	the gate above; policies.yaml

No limits declared is an error, never permission. A resource written by any binding must name an interlock policy. A profile may state explicitly that a resource has no limits, but silence is a compile failure.

MHS decides no authority. A mutation needs an admission: a hash of capability and parameters, an expiry and a signature, minted by a gate that is not MHS. The core verifies through a pluggable verifier and cannot mint. With no verifier configured, as in a fresh checkout, every mutation is refused: the layer fails closed.

The model never closes a loop. Continuous control is a controller declared in the registry with its kind, period, limits and safe state. The model sets a goal through a setpoint capability and observes the result through a lens.

Every refusal names its reason. A refusal is a typed result with a stable code. A generic error or denied is never used, because the code is the evidence and a vague code is an unusable record. A code is added with its enumeration entry, its traceability row and a test that provokes it; a code no test provokes has never been shown to fire.

IV. How Agency and the Registry Help Each Other

An agent and a registry are complementary in a specific way: the agent supplies intent in the vocabulary of the task, and the registry supplies everything the intent must not be trusted to know. Table II pairs each need an agent has with the registry logic that meets it.

Meaning in, addresses never out. A model plans in words—"is the cabin too warm", "locate the car"—and the semantic resolver maps words to capability ids. The binding resolver then picks the path. The model never learns that the cabin temperature is a broker path, a bus frame or a register on a serial sensor, and cannot ask for one, because the compile rules keep native text out of every string it can see. This is what makes tool schemas

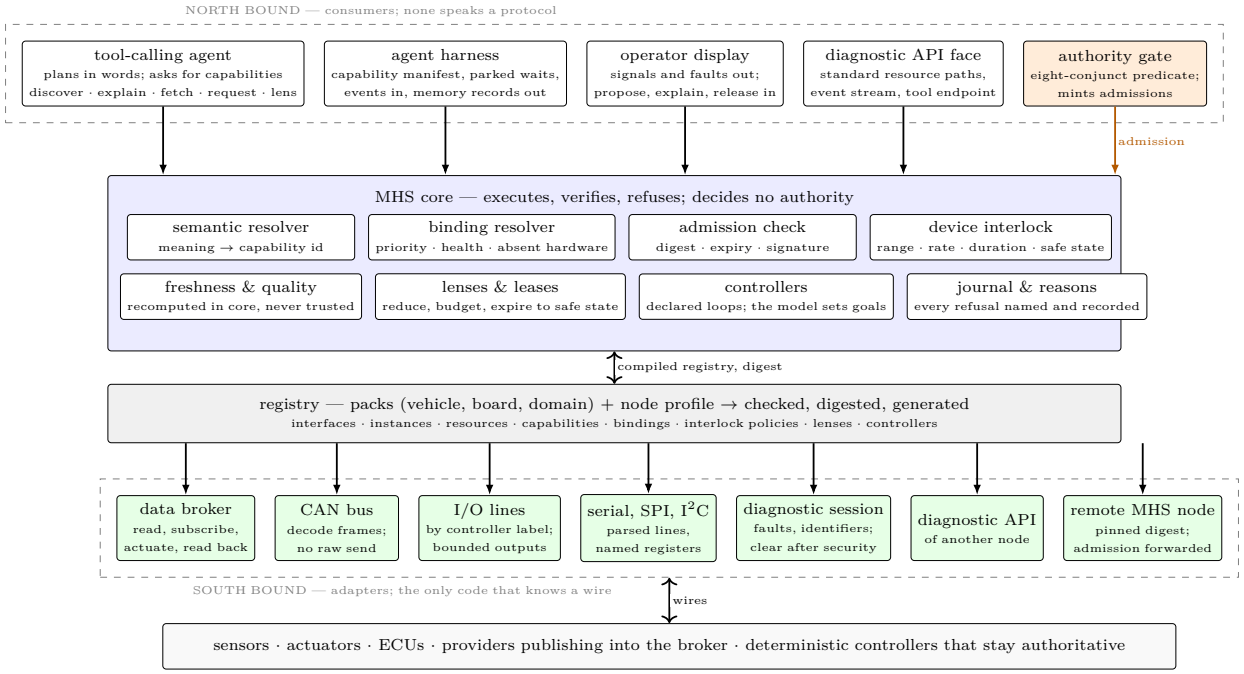


Fig. 1. North bound and south bound. Consumers above the core ask in the vocabulary of capabilities and receive typed results or named refusals; adapters below the core are the only code that knows a wire. The registry between them is data, and every arrow through the core is checked against it. Only the gate can produce an admission, and an admission is verified in the core, never minted there.

small and stable: a new sensor adds a capability, not a tool.

Bounded semantics make plans checkable. Because every capability carries a type, a class, a safety level, a quantity and a unit, a plan can be checked before it runs. The dry run answers with the same decision and the same codes as the real path, so an agent can reason about a refusal it has not yet received—and a harness can decide whether to ask a human before the request is ever made.

Fallback without surprise. A fallback binding must measure the same quantity, be no less fresh and have a distinct priority, and a mutation may have exactly one path. An agent therefore never sees a number whose meaning changed because the usual path was down, and never triggers an action through a path it did not intend.

The generic core is what makes agent skills portable. An agent skill written against the fourteen tools runs unchanged on a vehicle node, a bench node, a compute node that federates them, or a future robotics or industrial node, because the core knows none of them. What differs between nodes is data: the packs and the profile. The boundary test keeps it so.

Refusal is a first-class result. An agent that only ever hears “done” cannot learn its limits. Every refusal here names its cause, and a refusal is journalled like a success. This closes the loop the other way: an unexercised refusal becomes a test, and a test becomes a rule.

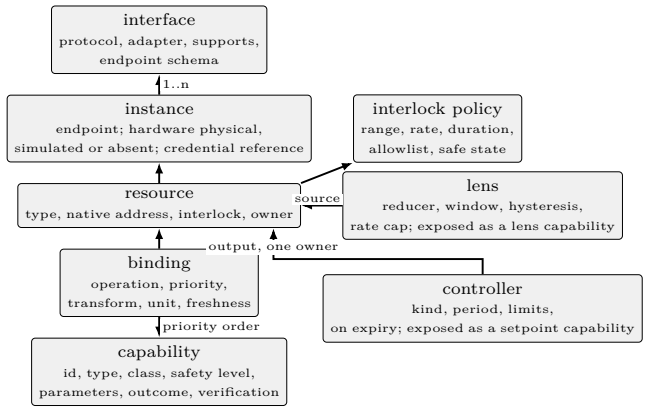


Fig. 2. Registry records. The load-bearing edge is binding → resource: a capability reaches hardware only through a binding, and only the resource’s native block knows the address.

V. The Registry

A. Records

Figure 2 shows the eight record types of one node profile. An interface is one technology: its protocol, the adapter that speaks it, the operations it supports and a schema for its endpoints. An instance is one bus, port or server, with an endpoint validated against that schema, a hardware state that may be absent, and credentials as a reference resolved at the moment of use from the environment or a secrets store and never written to a result, an event or a log line. A resource is one native item

TABLE II
What an Agent Needs and What the Registry Supplies

Agent need	Registry logic
name a thing by meaning, not by address	semantic resolver over ids, descriptions and aliases; native addresses never returned
know what is available now	instances carry a hardware state; the binding resolver skips absent or unhealthy paths
trust a number	unit and quantity on every capability; freshness recomputed in core; a value without a timestamp is refused
rehearse before acting	explain: the same decision path, nothing executed, the same reason codes
act only within its authority	a mutation needs an admission for its exact digest; the core verifies and cannot mint
never harm a device	interlock policy on every written resource; a refusal under a separate name
watch without drowning	lenses reduce at declared rates under a byte budget; events carry sequence and quality
hold something safely	leases expire to a safe state; a stopped agent leaves nothing held
control without closing a loop	declared controllers; the agent sets a goal and reads a lens
see many nodes as one	a remote node is an interface with a pinned digest
be told why	every refusal names its reason; a code no test provokes does not exist
work on another platform	packs carry the platform; the same fourteen tools and the same verbs everywhere

on an instance. A capability is what an agent may name: a type (state, command, event, query, diagnostic, alert, control, job, stream, lens or setpoint), a safety level S0–S4, an autonomy class C0–C3, a lifecycle (active, declared, refused or proposed), a parameter schema, an outcome schema, a quantity and unit, a freshness bound and, for a mutation, how it is verified. A binding maps a capability to a resource with an operation, a priority, an input mapping, a linear transform and a unit. Lenses and controllers are described in Section VIII.

The type fixes the verb an agent uses: states, queries and read-only diagnostics answer a fetch; commands, controls and mutating diagnostics need a request; a setpoint needs a set; a lens is reached by a subscription. A stream compiles only if the profile admits streams, and a lens over any camera-typed resource under a prefix the pack marks frame-free is refused, so no image bytes have a path to the model.

B. Packs and profiles

A pack is reusable data imported by a profile: interface definitions, capabilities in a prefix the pack owns, board facts, and units it adds to the core ontology. A pack has no instances, policies, resources or bindings, because those describe one node. A profile may tighten a pack capability—stricter lifecycle or class, added aliases or preconditions—and may not loosen, rename, remove or change its quantity, unit or type. Two packs cannot own one prefix. The digest covers packs and profile together, so a pack change is a registry change. The bench profile imports a vehicle pack (the signal catalogue; data-broker, CAN, diagnostic-session and diagnostic-API interfaces; the vehicle and cabin capabilities, including those refused by name), a board pack (the I/O controller's label, reserved lines, the lines each bus occupies, device patterns for the probe) and a small lamps pack that shows a domain is only data.

The core ships an ontology of fourteen quantity kinds—speed, temperature, ratio, voltage, current, power, energy, frequency, duration, length, angle, pressure, count, boolean—each with its SI unit and a conversion table. A capability with a unit names its quantity, and a fallback binding may answer in another unit of the same quantity; the core converts. A pack may add units to a kind; no pack may redefine a factor or add a kind.

C. Compile rules

Table III lists the checker's sixteen rules. Each has a mutation test that edits a copy of the bench profile and asserts the compiler goes red with the expected sentence, so a rule that stops firing is a failing test rather than a quiet gap. Every finding says which rule broke and what it would have caused.

D. Adding hardware

A device enters as data: instance, policy, resource, capability, binding, in that order, with a scaffold command producing the skeleton and the checker run until clean. A new technology also needs an interface definition; a new protocol needs an adapter—the only case that needs code. Hardware can also propose itself: a probe enumerates the operating system's device tree, scans I^2C addresses read-only, lists the broker's signal metadata and the diagnostic API's components, and writes instances and resources in the proposed lifecycle. A proposed row compiles, appears in status, and never appears in tools, discovery or description until a human promotes it. The diagnostic tester-present sweep exists and is off by default because it touches ECUs.

E. Digest, reload, secrets

The digest is recomputed on every compile. A running core hot-swaps a new registry atomically; open leases on capabilities whose contract changed are released with REGISTRY_CHANGED. Section IX uses the same digest to pin a remote node.

VI. Core and Adapters

A. Adapter contract

An adapter implements a fixed set of operations—initialise, health, read, execute, query, subscribe, poll,

TABLE III
Compile Rules, Each Tripped by a Mutation Test

Rule	What it prevents
endpoint validates against the interface's endpoint schema	a port opened at an impossible baud rate
no secret-like key in an endpoint; credentials are references	a bearer token in a reviewed file
GPIO by chip label, not on a bus or reserved line	the wrong chip on a 2024 kernel; a button on the I^2C clock
C3 is always refused; class $\geq$ minimum for its safety level	a C3 capability in a manifest
an active state has a freshness bound; an active C2 mutation says how it is verified	a value that can never be stale; an unverified effect reported as done
verification is by an active read capability	a read-back that is itself a write
params: object schema, no root union, no grammar-hostile keyword	a 400 for the whole session; a grammar looser than the schema
agent-visible text names no VSS path, device node, hex id, secret or chip label	a refusal sentence leaking a raw frame identifier to the model
no binding on a refused or declared capability, or an owned resource	a path to hardware for a name that is refused
operation supported by the interface and allowed for the verb	a command reached through a read
broker path in the signal catalogue; actuate only an actuator; unit matches	writing a sensor; km/h answered as mph
every write names an interlock; bus writes bound the value	a write nothing bounds
exactly one execute path per mutation	a silent fallback changing who acted
fallback: same quantity, freshness no looser, distinct priority	a fallback that changes what the number means
a direct fallback reads the wire the catalogue says feeds the broker	an I^2C fallback reading register 3 while the provider reads 2
broker version equals the catalogue pin	testing one broker, shipping another

start a job, shut down—and advertises the subset it supports. Every adapter takes its bus, port, chip or transport as an injectable backend, so its logic runs in the pure-logic test tier and the real device path is a hardware tier. Every write names a command or a bounded value; none accepts free bytes. A serial resource names a parser, and only a line the parser accepts becomes a value: an NMEA sentence with a bad checksum is not a position. A free-text serial write is the serial equivalent of raw frame injection and is not expressible.

Table IV lists the wires the packs reach. Each result carries the kind of timestamp it has, because a value without a timestamp cannot be judged fresh and is refused (TIMESTAMP_MISSING), as is one more than a second in the future (CLOCK_SKEW). A device's negative response—a diagnostic negative response code, an HTTP client error—becomes DEVICE_REFUSED with a fixed sentence; the native code goes to the operator log only.

TABLE IV
Wires the Vehicle and Board Packs Reach, and the Rule for Writing to Each

Wire	Timestamp	Write rule
data broker	broker update time	actuate only a signal the model marks as an actuator; read-back mandatory
CAN bus	source sample time	none: raw frame send is refused by name; a physical bus only when approved
I/O lines	edge time	output lines only; the policy bounds the value
serial port	adapter read time	a named command from the resource's list; a parsed line, never raw bytes
SPI device	adapter read time	a named register write
I^2C device	adapter read time	a named register write; raw counts scaled by the binding
diagnostic session	adapter read time	clear faults only, after security access with a stored secret
diagnostic API	adapter read time	the one path the resource names; token from the store
remote node	the remote's evidence	the admission is forwarded; the remote decides

B. Resolution and a read

Discovery is a semantic resolver over capability ids, descriptions and aliases; execution is a binding resolver that tries bindings in priority order and skips an instance that is absent, unhealthy or proposed. On a read the adapter returns a value and its timestamp; the core applies the binding's transform and unit conversion, recomputes freshness against the capability's bound rather than trusting the adapter, and reports which binding answered and which were tried. Reads fall back; writes do not, because a silent fallback on a mutation would change who acted. Both paths of a diagnostic read answer in one shape, so a fallback does not change the meaning of the answer: a store that cannot be read is a refusal, never an empty list.

C. Subscription-fed state

Where an interface supports it, the core subscribes once and serves reads from the cache with the source recorded as a subscription and the age computed from the broker's own stamp; a path the broker does not know stays polled. A background retry revives an interface that went away without restarting the process.

VII. Two Locks: Authorisation and Interlock

Figure 3 is the control path. The gate is the ACG of [3], which admits a request q only if

$$\begin{aligned}\mathrm{Allow}(q) = \mathrm{Auth}(i,t) \wedge \mathrm{Schema}(x) \wedge \mathrm{Fresh}(s) \\ \wedge \mathrm{Pre}_t(s) \wedge \mathrm{Risk}(t,s) \leq \mathrm{Permit}(i) \\ \wedge \mathrm{Consent}(q) \wedge \mathrm{Budget}(q) \wedge \mathrm{Before}(\tau),\end{aligned} \quad (1)$$

and, when it does, mints an admission

$$a = \langle \mathrm{H}(t \,\|\, x),\ \tau_a,\ \sigma \rangle \quad (2)$$

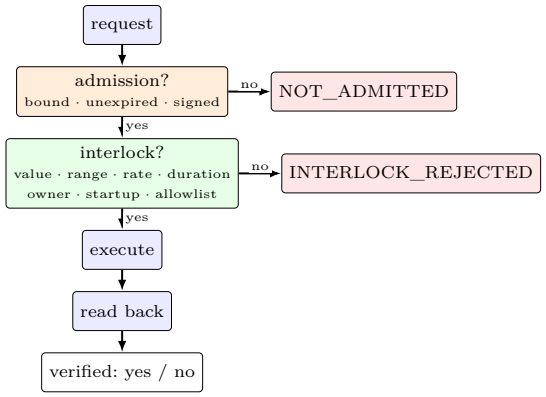


Fig. 3. Two locks. The load-bearing edge is yes → interlock: an action the gate admitted can still be refused at the device. The gate decides whether this principal may ask now; the interlock decides whether this device may do this at all, regardless of who asked.

bound to the digest of the exact capability and parameters, with an expiry and a signature. MHS receives a with the request, checks the digest, the expiry and the signature through a verifier that delegates to the platform's trust anchor, and refuses with NOT_ADMITTED otherwise. It never evaluates Allow and holds no key that could produce σ. The gate's own reason codes—RISK_EXCEEDS_PERMIT, REDACTION_UNAVAILABLE, NO_WRIT—do not exist in MHS, because MHS decides no authority.

The second lock is the interlock policy named by the resource. It bounds the value (a set or a range), the rate, the duration of a held output, the commands a serial or register write may name, whether a write is allowed in the first seconds after boot, how many leases may hold the resource and for how long, and the value the resource returns to when a lease expires or a loop halts (the safe state). A line held by another process cannot be bound at all. The interlock is the "safety limits enforced at the device" of [4], and its separation from the gate is the point: two checks, two names, two codes.

Safety level and class are different questions. The level asks how bad the effect is if wrong and is set by the hazard of the device; the class asks what authority is needed to ask and is set by the platform's autonomy policy. A class may be stricter than its level, never looser: S0→C0, S1→C1, S2→C2, S3/S4→C3, and C3 is always refused. Door unlock is S2 in the original design and held at C3, refused by name, in the vehicle pack. The refusal is two walls deep: the capability is in no manifest, and the central-lock resource's interlock allows the locked and secured values only, so even a capability that asked for the unlock direction could not put it on the bus.

A request in flight is keyed on principal, capability and request digest. A second identical request while the first runs is answered ALREADY_IN_PROGRESS rather than executed twice; a completed one within the freshness window is answered from the cached outcome. An admitted mutation is journalled before it executes, and the outcome names the binding that acted and whether the read-back matched.

VIII. Live Data, Leases and Controllers

High-rate data never crosses to the model; it is reduced first. A lens is a declared reduction over a resource or another lens: sample, window (min, max, mean, last, count over a window), threshold (with mandatory hysteresis) or change (with debounce). Four reducers, fixed in core, no expressions and no user code. The profile bounds every lens's rate at compile time; the graph of lenses must be acyclic. Egress is a bounded queue per subscription that drops oldest and counts what it dropped, under a byte budget per minute; exceeding the budget refuses the next drain with BUDGET_EXHAUSTED and releases the lease. Every event carries a sequence number, the time it occurred, the time it was observed, the window bounds and a quality mark.

Anything held over time is a lease: acquired with a time-to-live bounded by policy, renewed not before half its life, released explicitly. Expiry ends a lens subscription and returns a controller's output to its safe state. A principal holds at most a configured number of leases (LEASE_LIMIT); a renewal after expiry is LEASE_EXPIRED.

A controller is a deterministic loop declared in the registry: a kind (bang-bang, PID, schedule or bounded pass-through), an output resource that must have an interlock and exactly one owner, a setpoint capability, a period no shorter than the profile allows, output limits and slew, and what to do on lease expiry or stale input. The runner writes its output through the same interlock path as request; a write of exactly the policy's safe state bypasses the rate and startup limits so that a halt is never refused. Stale input halts the loop and emits CONTROLLER_HALTED on the controller's built-in status lens. The invariant a controller keeps is

$$\mathrm{out}(k) \in \mathrm{Range}_{\mathrm{pol}} \ \wedge \ \mathrm{live}(\ell) \ \wedge \ \mathrm{fresh}(\mathrm{in}(k)), \qquad (3)$$

and when any conjunct fails the output is the safe state before the next tick. Because the model only ever sets a goal and reads a lens, a model that stops, hallucinates or is disconnected leaves a loop that either continues correctly or halts safely.

IX. Federation

A remote MHS node is an interface like any other: a remote protocol, an endpoint naming the remote's address, node id and a pinned digest, and resources of a remote-capability type whose native block names the remote capability. The remote's own description supplies quantity, unit and freshness, and the compiler checks quantity equality with the local capability. The local digest is $\mathrm{H}(\mathrm{local} \,\|\, \mathrm{remote})$, so a remote change is a local registry change. A pin mismatch keeps the adapter ready

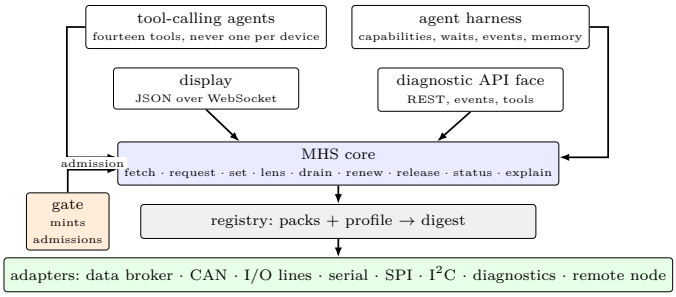


Fig. 4. One core, many faces. No consumer speaks a protocol; every consumer reaches hardware through the same verbs, and only the gate can produce an admission.

and answers every call REGISTRY_CHANGED until an operator re-pins after reading the change; an unready adapter would have been skipped by the resolver and surfaced as a generic unavailability. An admission travels with the call and is verified by the remote against its own registry, so a remote mutation must keep its capability id. A local lease wraps a remote lease, and remote expiry surfaces as LEASE_EXPIRED. On the bench, a compute node (Jetson) imports the vehicle node (Raspberry Pi) this way, and a remote value's liveness comes from the remote's evidence.

X. Agent Surface and Consumers

An agent sees fourteen tools and never one per capability: four that read the registry and carry no authority (discover, describe, status, explain); one that reads a value (fetch); two that mutate and need an admission (request, set); four that manage leases (subscribe to a lens, drain, renew, release); and two deprecated aliases kept for one release. Each tool's schema holds one call property whose alternatives are the active capabilities, each paired with its own parameter schema, so an unadmitted pairing is unrepresentable and the alternatives are not at the schema root where the model APIs reject them. A build step generates the constrained-decoding grammar and fails on any warning, and the compiler refuses the keywords the converter skips. Explain is a dry run: the gate, the binding order with availability, the policy, whether an admission is needed, what would verify, and a decision of would allow or would refuse with the same reason codes; nothing executes.

Figure 4 shows the consumers built so far. An agent harness reaches MHS through four seams—its capability manifest, a worker that returns a deferred handle while the gate parks a request for a writ or physical presence, lens events posted as harness events with MHS as the actor so that a controller-write-then-lens loop cannot restart a run, and results written as memory records with a validity end at observed time plus freshness. A cabin display speaks JSON over a WebSocket, receives signals and fault stores, and proposes, explains and releases; disconnection releases every lease it held. A standard service-oriented diagnostic server [12] presents the same core as a diagnostic API—the sixty standard resource paths, server-sent events and a tool endpoint—with its entities as data rather than code; it mints admissions for C0 and C1 only, and a C2 request needs a writ from the gate. Each consumer is a different face on the same verbs.

TABLE V
Reason Codes MHS Emits and What Provokes Each

Code	Provoked by
NOT_IN_MANIFEST	unknown, declared-not-built, or refused by name
SCHEMA_INVALID	parameters fail the capability's schema
STALE_EVIDENCE	older than the binding's freshness bound
SOURCE_UNAVAILABLE	no healthy binding; hardware absent
DEADLINE_EXCEEDED	the adapter timed out
NOT_ADMITTED	no valid admission for this digest, or the wrong verb
BUDGET_EXHAUSTED	a lease spent its egress budget; it is released
RATE_LIMITED	renewed too early, or asked faster than a bound allows
CLOCK_SKEW	timestamp more than 1 s in the future
TIMESTAMP_MISSING	a value with no timestamp
INTERLOCK_REJECTED	the device policy refuses an admitted write
DEVICE_REFUSED	a negative response; the native code stays in the log
LEASE_EXPIRED	renewal after the time-to-live; remote lease expiry
LEASE_LIMIT	one lease more than the principal may hold
CONTROLLER_HALTED	stale input; set on a halted controller
REGISTRY_CHANGED	pin mismatch; hot reload under an open lease
ALREADY_IN_PROGRESS	same principal, capability and digest still running

XI. Reason Codes

Table V lists the codes. The upper group shares names and meanings with the gate; the lower group is emitted on the node, below the gate's decision. Each lower code has a pure-logic test that provokes it, and a boundary test asserts that the codes in code are a subset of the codes of record.

XII. Generic Across Platforms and Domains

The answer to RQ3 is the composition in Figure 5 and the test that enforces it. The boundary test walks the core's source and asserts: no import of a host application's package; protocol libraries only inside the adapters; only one file speaks the data broker's API; no string in the logic names the bench, a vehicle make, a signal path, a bus-database signal prefix, a controller label or a serial device; no flash path anywhere; the harness bridge imports no adapter and the adapters import no bridge; no device path in the probe. When the probe needed device globs, the test refused them in core and they moved to the board pack's interface definitions. When a first probe draft on a different board re-proposed a bus the profile already declared, the draft was restricted to present but undeclared. The pattern is that each attempt to put board or domain knowledge

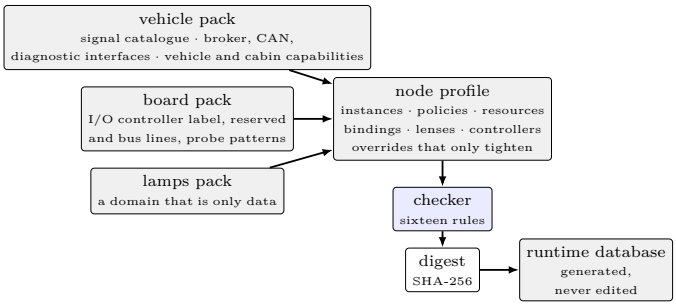


Fig. 5. Composition. Packs carry domain and board knowledge; the profile carries one node; the core carries neither. A future robotics, industrial-bus or instrument pack adds interface definitions, an adapter and capabilities in its own prefix, and core does not change.

in core has been caught by a test and answered with a pack field.

A worked example from the design documents makes the cost of a new device concrete: a temperature sensor on I^2C and a relay on an I/O line are one instance each, one policy for the relay bounding its value and rate, two resources whose native blocks hold the address, register and line, two capabilities in the domain's prefix with quantity, unit and freshness, and two bindings. No code. The same profile, moved to a board without an I^2C bus, marks the instance absent; the capability stays declared, the resolver skips it, and a request answers SOURCE_UNAVAILABLE rather than a fabricated reading.

Not everything has left core yet. The vehicle signal catalogue is still mandatory at compile time, the frame-free prefix and a few native-text patterns are literals, the I/O protocol is named for the board, and the display bridge names one diagnostic capability. Each is documented as a work package with its acceptance criterion—the generated agent surface for the bench profile must remain byte-identical—and each moves a fact into the vehicle or board pack and leaves a declarative slot in core. The direction is set by the tests already in place; the remaining moves are mechanical.

XIII. Evaluation

A. Test tiers

Every test states its tier, and a tier that cannot run announces a skip with its reason, never a silent pass. Tier 1 is pure logic over injected backends and runs anywhere: registry compile and the sixteen mutation gates; pack import and override rules under twelve mutations; ontology round trips; reducers; lease life cycle and limits; byte budgets; a controller reaching its setpoint over a fake adapter and halting on stale input; the dry run's decision equal to the real decision on every bench capability; the harness bridge validating one hundred random parameter sets identically to the registry; and the boundary test. Tier 3 needs a live broker, a virtual CAN bus or an ECU simulator; Tier 4 needs the agent harness on the path; Tier 5 runs two nodes federated in-process. On the extracted repository at the time of writing, Tier 1 is 308 tests passing with one reasoned skip, Tier 5 passes, and Tiers 3 and 4 skip with their reasons printed.

TABLE VI
Single-Run Measurements on the Bench (Raspberry Pi 5 Vehicle Node, Jetson Compute Node)

Path	Measured
diagnostic API read of a subscribed signal, on the node	2.3–2.6 ms round trip; 0.8–1.1 ms inside MHS
diagnostic API read from the compute node (WLAN, keep-alive)	7.8 ms
faults over a diagnostic session on a virtual CAN bus	6 ms on the node; 12.6 ms remote
C1 setpoint over a diagnostic session, read back from the ECU's 1 Hz frame	140–325 ms, verified: true
C2 request with no admission	NOT_ADMITTED
two identical writes in flight	one 200; one ALREADY_IN_PROGRESS in 17 ms; one effect
two identical I/O-line writes	one 200, one INTERLOCK_REJECTED (5 Hz bound)
agent read over the tool protocol from the compute node	p50 9.7 ms
agent's first read (session, pin, fetch)	102 ms
display touch to lamp, verified by read-back	2.6 ms
state cache	32 subscriptions on one broker stream

B. Bench measurements

Table VI reports what was measured on the bench: a Raspberry Pi 5 vehicle node holding the I/O lines, an ECU simulator transmitting on a virtual CAN bus at 302 frames per second and answering diagnostic requests, a data broker fed by a CAN provider, and a Jetson compute node over WLAN. The bench profile compiles to 8 interfaces, 7 instances, 89 resources, 115 capabilities of which 101 are active, and 100 bindings. These are single runs and are reported as such; no confidence intervals are claimed. Two of the rows are refusals, deliberately: a table of latencies in which every ceiling fires zero times looks identical to one in which no ceiling can fire.

C. What the evaluation shows and does not

The measurements show that the two-lock path, the in-flight table, the read-back and the subscription-fed cache behave as designed on real sockets, a real I/O controller and a real diagnostic session, and that the per-read cost inside MHS is about a millisecond. They do not show behaviour under load, over time, or against a real vehicle. The evaluation of the lens, lease, controller and federation designs is at Tier 1 only; those parts have met injected backends and, for federation, two in-process nodes, but no hardware.

XIV. Limitations

The following are stated so that the reference implementation does not overclaim. The standard is unpublished. Every statement about the Model Hardware Standard rests on an announcement; alignment is by shape and vocabulary, and conformance is not claimed. When a specification is published, the comparison must be redone.

No hardware for lenses, controllers or federation. Parts of Sections VIII and IX are proven over injected backends and two in-process nodes only.

An actuation with no provider. on the deployed broker has not been observed; the test that would settle it skips without a broker, and the reason-code mapping waits for the answer.

The interlock is in-process. A compromised node process is a compromised interlock; the interlock protects against a wrong request, not a wrong node.

One trust anchor. Federation assumes a shared trust anchor between nodes; key rotation is not designed.

Lenses reduce; they do not understand. A one-second mean hides a fifty-millisecond spike, and the threshold reducer catches spikes only at declared thresholds.

Four controller kinds; gains are authored and untuned. There is no auto-tuning.

Fourteen quantity kinds. A pack that needs a new kind edits core. This is deliberate—one ontology—and it is a limit.

The diagnostic API face is not certified. Its sixty paths match the standard's list by name; there is no authentication on it and no service discovery.

The vehicle was simulated. The bench ECU is a simulator; the CAN bus is virtual on the node; the lamps are lamps.

Some vehicle knowledge remains in core., as listed in Section XII, with the acceptance criterion for each move.

Single-run numbers. Table VI has no repetitions and no confidence intervals.

XV. Conclusion

A hardware layer for agents can be data. MHS states every device, capability, fallback, freshness bound and safety limit in a registry that a compiler checks with rules that are each shown to fire, and serves it from a core that a test prevents from learning any vehicle, board or pin. It executes without deciding authority—an admission is verified, never minted—and refuses on its own account only when a device's declared limits say so, under a separate name. Live data is reduced before it leaves, anything held is leased, control loops are declared and the model only sets goals, and nodes compose by pinned digest. Domain and board knowledge arrive as packs, with the vehicle first. What is measured is reported with its conditions; what is not is listed. The result is a layer that a gate such as the one in [3] can sit on, and that other domains can adopt by writing a pack rather than a driver.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[3] Yuvaraja, "A vehicle-first agentic AI architecture for software-defined vehicles." Preprint, IEEE conference format, 2026. Companion paper; defines the Automotive Capability Gateway and the invariants MHS sits under.

[4] Anthropic, "Previewing the model hardware standard." Research preview announcement, 27 August 2026, 2026.

[5] Model Context Protocol Contributors, "Model context protocol specification." Revision 2026-07-28, 2026.

[6] AUTOSAR, "Specification of communication management," Tech. Rep. Adaptive Platform R25-11, AUTOSAR, 2025.

[7] COVESA, "Vehicle signal specification." Online specification, 2025.

[8] Eclipse KUKSA Contributors, "KUKSA databroker user guide." Eclipse KUKSA, version 0.7, 2026.

[9] International Organization for Standardization, "Road vehicles—unified diagnostic services (UDS)—part 1: Application layer." ISO 14229-1:2020, 2020.

[10] International Organization for Standardization, "Road vehicles—diagnostic communication over controller area network (DoCAN)—part 2: Transport protocol and network layer services." ISO 15765-2:2016, 2016.

[11] International Organization for Standardization, "Road vehicles—diagnostic communication over internet protocol (DoIP)—part 2: Transport protocol and network layer services." ISO 13400-2:2025, 2025.

[12] ASAM e.V., "SOVD: Service-oriented vehicle diagnostics." Standard overview, 2024.

[13] International Organization for Standardization, "Road vehicles—service-oriented vehicle diagnostics (SOVD)." ISO 17978 series, 2025.

[14] Raspberry Pi Ltd, "A history of GPIO usage on Raspberry Pi devices and current best practices." White paper RP-006553-WP, 2024.

[15] B. Golaszewski and contributors, "libgpiod: C library and tools for interacting with the Linux GPIO character device." Version 2.x, 2025.

[16] Kolonie AI, "MCP tool schemas with root-level oneof/allof rejected by the tool-use API." Issue tracker, kolonie-platform #1964, 2026.

[17] ggml contributors, "llama.cpp GBNF grammars and JSON schema conversion." Project documentation, 2026.